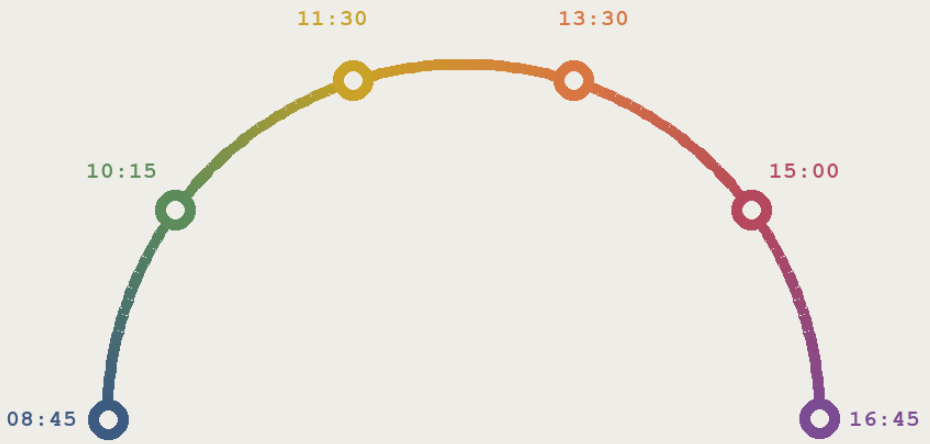
CLINICAL PROGRAMMING BOOTCAMP · DAY 1

A day in the life

What a clinical programmer actually does, hour by hour



Before we open a single dataset — this is the job you are training for.

The whole job, in one sentence

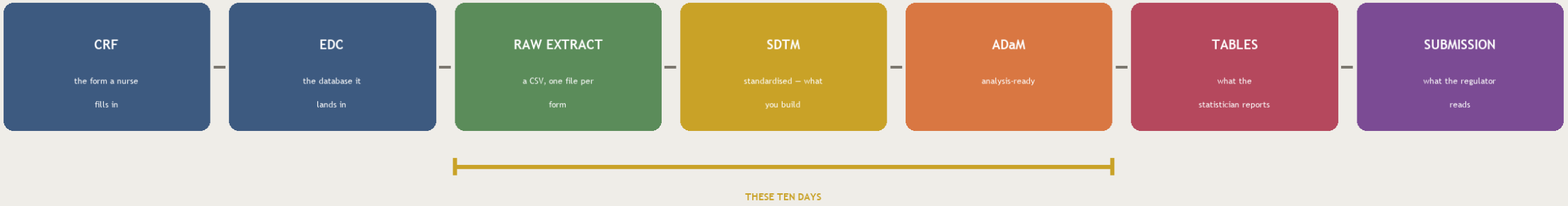
> Take the messy data a hospital actually recorded, and turn it into something a regulator on the other side of the world can read without asking you a single question.

Everything else — the standards, the code, the checks — exists to make that sentence true.

You are one link in a long chain

Where your work sits

Every dataset you build has a nurse at one end and a reviewer at the other.



A nurse writes on a form. Years later a reviewer decides whether a medicine reaches patients. Your datasets are what connects those two people.

The ten days ahead cover the amber stretch — raw extract through to analysis-ready.

08:45

The first thing I do is read yesterday's log

Not the output. The log. Overnight the study data refreshed and every programme re-ran.

Why this comes first

A programme can finish with no error and still be wrong.

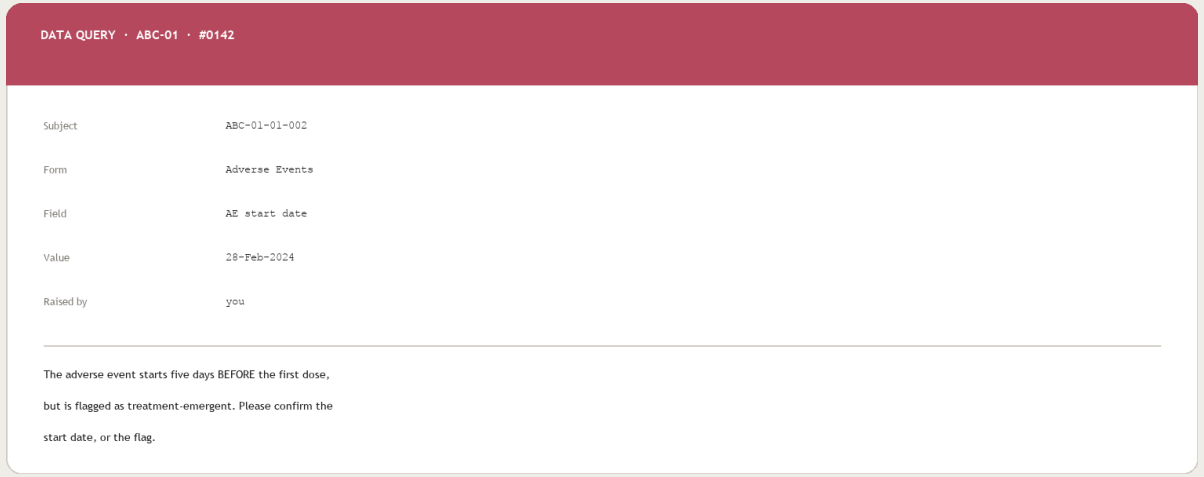
Three of the four worst bugs in this course announced themselves only as a WARNING.

“It ran fine” is not a status. The log is the status.

10:15

The data asks me a question I cannot answer

Something in the data is not possible. I am not allowed to guess — so I ask the site.



The rule

A programmer never invents a value.

If a date is impossible, you do not repair it quietly. You raise a query, and the site — the people who saw the patient — answer it. That answer is part of the trial record.

Guessing is the one unforgivable habit in this job.

11:30

Now I build the thing I am paid to build

One raw file in. One standard dataset out. This is the craft at the centre of the iob.

What the site collected

SITEID	SUBJID	BRTHDTC	SEX
01	001	14-MAY-1969	2
01	002	02-NOV-1963	1
02	001	30-AUG-1977	2

you

What the reviewer receives

USUBJID	AGE	AGEU	SEX
ABC-01-01-001	54	YEARS	F
ABC-01-01-002	60	YEARS	M
ABC-01-02-001	46	YEARS	F

- subject 001 appears at BOTH sites
- sex is a CODE, not a word
- date of birth is collected

- one key, unique across the whole study
- code decoded to CDISC terminology
- date of birth is NOT submitted - AGE is derived from it

13:30

Half of this job is agreeing what a column means

Afternoon: a spec review. Five people, one spreadsheet, one argument about a variable.

What gets argued about	Whether a value is 'derived' or 'collected'. Where a non-standard flag belongs. Which visit counts as baseline.
Why it matters more than the code	The specification is the contract. Code that perfectly implements a wrong spec is wrong.
What you contribute	You are the person who has actually looked at the data. That makes you the person who knows what is really in it.

15:00

Someone else writes my programme again, from scratch

Double programming. Two people, one specification, no conversation — then the results are compared.

You

write dm.sas from the spec

Your QC partner

writes it again, never seeing your code

`PROC COMPARE`

If the two datasets differ by a single value, one of you is wrong — and the point is to find out which before a regulator does.

15:00

16:45

And then something changes

Late afternoon. The spec is updated, or the data refreshes, and work you finished is suddenly out of date.

> This is not a sign that something went wrong. It is the normal condition of a live study.

Which is why everything you will learn here is built to be RE-RUN, not hand-edited:

- **a programme, not a spreadsheet edit** — so the change is repeatable
- **a specification, not a memory** — so the reason survives the person
- **a check that fails loudly** — so a stale result cannot pass quietly

Where the day actually goes

The surprise for most people: writing new code is the small part.

Indicative of a typical study week — not a timesheet.

If you only remember one slide from today, remember this one.

You will spend far more time reading — code, logs, specifications, other people's programmes — than writing. Being fast at typing is worth very little. Being careful when reading is worth almost everything.

What actually makes someone good at this

None of these is about how much SAS you know on your first day.

- **Suspicion** — You assume a clean-looking result is hiding something, and you go and check.
- **Precision about words** — You notice that 'severe' and 'serious' are different, and that the difference matters.
- **Comfort asking** — You raise the query rather than quietly making the data look tidy.
- **Leaving a trail** — Someone can pick up your work in two years and see why every value is what it is.

All four are learnable. That is the good news.

What the next ten days look like

You will build a complete study — then do it again on your own.

Days 1–3	**The ground**	trials, the toolkit, and getting raw data in without breaking it
Days 4–7	**The domains**	DM, AE, CM, EX, VS, LB — one observation class at a time
Days 8–9	**The rigour**	controlled terminology, derivations, QC, and the submission package
Day 10	**On your own**	a second study, no worked notebook, four traps nobody solves for you

By Day 10 you will have built two studies, and read a lot more code than you wrote.

One thing to carry into tomorrow

A clean-looking run is not a correct run.

Everything in the next ten days is a way of proving that a result is right, rather than hoping it is. Bring the suspicion. We will supply the rest.

Tomorrow: Module 01 · Introduction to CDISC & SDTM Foundations